claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf_b7edcaa8-69d\agent-a9222a7d18a828442.jsonl`
- SHA-256: `b3d8a7462351b7b21766ac9ec177f3ad02bb09e2e55a31df0db43be33c666a3d`
- Source modified: `2026-06-21T14:36:45+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf_b7edcaa8-69d`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T14:33:51.170Z)

Spec the **web/ SPA UX tickets A4, A5, A8** for the Studio SPA in C:/Users/avidu/Projects/khelsutra. Read web/src/App.tsx, web/src/components/{QueryBar,RallyGrid,SelectionFooter,CompileResult,IngestPanel,LanguageSwitcher}.tsx, web/src/lib/errors.ts (already has classifyError), web/src/index.css, web/src/i18n/locales/en/common.json, web/src/i18n/hygiene.test.ts, and docs/FRIENDLY_AND_MULTILINGUAL.md §7 (A4/A5/A8 rows + §4 UX levers). Produce THREE tickets:
- A4 (mobile-first touch targets + make tooltip text visible on phones): exact min-height/tap-target Tailwind classes per interactive element, the tooltip?visible-text approach, file anchors.
- A5 (friendly engine-error mapping over raw FastAPI detail): the BUILD/compile path in App.tsx onBuild currently shows raw ApiError text for 5xx/network; reuse lib/errors.ts classifyError (already built for ingest) to map it to a friendly localized message in CompileResult ? specify a SHARED errors.* i18n namespace vs reusing ingest.* (recommend one), the exact onBuild change, and whether to unify IngestPanel onto the same namespace.
- A8 (background-processing reassurance + verdict/status as icon chips): where to add reassurance copy + render status as icon+text chips (never icon-only), the ingest progress + the build state.
For each: filesToTouch, changeSpec, newI18nKeys (en/kn/hi), testPlan, gotchas, effort. Note A4/A5/A8 all touch App.tsx + the catalogs so they must serialize. Return structured JSON (tickets:[A4,A5,A8]).

2. assistant (2026-06-21T14:33:53.594Z)

I'll start by reading the relevant files in the khelsutra project. Let me first ensure the repo is fresh and read all the specified files.

3. user (2026-06-21T14:33:57.462Z)

```
From https://github.com/avidullu/khelsutra
   a0475bc..a20002f  master     -> origin/master
Updating a0475bc..a20002f
Fast-forward
 docs/MULTILINGUAL_EXPANSION.md | 114 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 114 insertions(+)
 create mode 100644 docs/MULTILINGUAL_EXPANSION.md
---BRANCH---
master
---STATUS---
?? .claude/
?? site/access.db
```

4. user (2026-06-21T14:33:57.486Z)

```
1       import { useEffect, useMemo, useState } from "react";
```

```
2       import { useTranslation, Trans } from "react-i18next";
3       import { useRallySelection } from "./hooks/useRallySelection";
4       import { RallyGrid } from "./components/RallyGrid";
5       import { QueryBar } from "./components/QueryBar";
6       import { SelectionFooter } from "./components/SelectionFooter";
7       import { LanguageSwitcher } from "./components/LanguageSwitcher";
8       import { CompileResult, type BuildState } from "./components/CompileResult";
9       import { IngestPanel } from "./components/IngestPanel";
10      import { runQuery, type ParsedQuery } from "./lib/query";
11      import { floorRisk } from "./lib/floor";
12      import { reelFilename } from "./lib/format";
13      import { parseDetail } from "./lib/errors";
14      import { log, newCorrelationId } from "./lib/log";
15      import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "./api/client";
16      import type { EngineConfig, RallySegment } from "./api/types";
17
18      /** One compact, structured description of how a query was understood (for the trace
log). */
19      function parseTrace(q: ParsedQuery) {
20        return {
21          raw: q.raw,
22          recognized: q.recognized,
23          filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
24          sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
25          limit: q.limit ?? null,
26          unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
27        };
28      }
29
30      // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
31      // contract; only honest fields are surfaced. A real session loads from POST /api/query.
32      const DEMO_RALLIES: RallySegment[] = [
33        { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
34        { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
35        { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
36        { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
37        { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
38        { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
39      ];
40
41      type LoadState =
42        | { status: "demo" }
43        | { status: "loading" }
44        | { status: "loaded"; count: number }
45        | { status: "error"; error: string };
46
47      export default function App() {
48        const { t } = useTranslation();
49        // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
50        // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
51        // without a reload (it also rewrites ?video= so a refresh is stable).
52        const [videoId, setVideoId] = useState<string | null>(
53          () => new URLSearchParams(window.location.search).get("video"),
54        );
55
```

```
56        const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
57        const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
58        const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
59        const [reelName, setReelName] = useState("highlights");
60        const [build, setBuild] = useState<BuildState>({ status: "idle" });
61
62        // Display order is separate from selection: a query can re-sort the cards while the
shared
63        // selection set (sel) stays the single source of truth for what lands in the reel.
64        const sel = useRallySelection(rallies, "all");
65
66        // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
67        useEffect(() => {
68          if (!videoId) return;
69          const cid = newCorrelationId();
70          log("info", "rallies.load_start", { cid, video_id: videoId });
71          // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
72          // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
73          setLoadState({ status: "loading" });
74          let alive = true;
75          listRallies(videoId, cid)
76            .then((rs) => {
77              if (!alive) return;
78              setRallies(rs);
79              setOrder(rs);
80              sel.apply(rs.map((r) => r.id));
81              setLoadState({ status: "loaded", count: rs.length });
82              log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
83            })
84            .catch((err) => {
85              if (!alive) return;
86              setLoadState({ status: "error", error: String(err) });
87              log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
});
88            });
89          return () => {
90            alive = false;
91          };
92          // videoId is stable for the page; sel.apply is a stable callback.
93          // eslint-disable-next-line react-hooks/exhaustive-deps
94        }, [videoId]);
95
96        // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
97        // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
98        const [cfg, setCfg] = useState<EngineConfig | null>(null);
99        useEffect(() => {
100         let alive = true;
101         getConfig()
102           .then((c) => alive && setCfg(c))
103           .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
104         return () => {
105           alive = false;
106         };
107       }, []);
108       const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
109
110       // Stable chronological ordinal (by start_time) so re-sorting never renumbers a rally.
111       const ordinalOf = useMemo(() => {
112         const m = new Map<number, number>();
113         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
```

```
        return (id: number) => m.get(id) ?? 0;
      }, [rallies]);

      // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
      const onBuild = async () => {
        if (build.status === "building") return;
        const cid = newCorrelationId();
        const filename = reelFilename(reelName);
        const ids = [...sel.selected];
        if (!videoId) {
          log("info", "build.no_video", { cid, selected: ids.length });
          setBuild({
            status: "error",
            error: t("build.noVideo"),
          });
          return;
        }
        setBuild({ status: "building" });
        log("info", "build.start", {
          cid,
          video_id: videoId,
          filename,
          selected: ids.length,
          total_sec: Math.round(sel.totalSec),
          below_floor: warning?.atRisk.length ?? 0,
        });
        try {
          const res = await compileReel(videoId, ids, filename, cid);
          const url = highlightUrl(videoId, filename);
          setBuild({ status: "done", url, filename });
          log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
        } catch (err) {
          const status = err instanceof ApiError ? err.status : undefined;
          const detail = err instanceof ApiError && err.body ? parseDetail(err.body) :
String(err);
          setBuild({ status: "error", error: detail });
          log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });
        }
      };

      // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
      // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
      // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
      const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
        const cid = newCorrelationId();
        log("info", "query.parse", { cid, source, ...parseTrace(q) });

        // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
        // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
        if (q.unavailable.length > 0) {
          log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
        }

        if (!q.recognized) {
          log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
          return; // empty / unparseable ? leave the user's curation untouched
```

```
        }

        try {
          const { ordered, selectedIds } = runQuery(rallies, q);
          setOrder(ordered);
          sel.apply(selectedIds);
          log("info", "query.apply", {
            cid,
            source,
            filters: q.filters.length,
            sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
            limit: q.limit ?? null,
            total: rallies.length,
            selected: selectedIds.length,
            selected_ids: selectedIds,
            dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
          });
          if (selectedIds.length === 0) {
            // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
            log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
          }
        } catch (err) {
          // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
          log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
        }
      };

      // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
      const onToggle = (id: number) => {
        log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
        sel.toggle(id);
      };
      const onBulk = (op: "select_all" | "clear" | "invert") => {
        const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
        log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
        if (op === "select_all") sel.selectAll();
        else if (op === "clear") sel.clearAll();
        else sel.invert();
      };

      return (
        <main className="min-h-screen bg-paper text-ink pb-24">
          <header className="bg-ink text-white">
            <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
              <span className="text-xl font-extrabold tracking-tight">
                Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
              </span>
              <LanguageSwitcher />
            </div>
          </header>

          <section className="mx-auto max-w-5xl px-6 py-10">
            <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
              {t("app.badge")}
            </span>
            <h1 className="mt-4 text-3xl font-extrabold tracking-
```

```
tight">{t("app.title")}</h1>
            <p className="mt-2 max-w-2xl text-ink/70">
              <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
            </p>

            {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
                path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
            {!videoId && (
              <IngestPanel
                onLoaded={(id) => {
                  setVideoId(id);
                  window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
                }}
              />
            )}

            <ModeBanner load={loadState} videoId={videoId} />

            <div className="mt-6">
              <QueryBar onApply={onApplyQuery} />
            </div>

            <div className="mt-6 flex flex-wrap gap-2 text-sm">
              <button type="button" onClick={() => onBulk("select_all")} className="rounded-
full border border-black/15 px-3 py-1 hover:border-green">{t("bulk.selectAll")}</button>
              <button type="button" onClick={() => onBulk("clear")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">{t("bulk.clear")}</button>
              <button type="button" onClick={() => onBulk("invert")} className="rounded-full
border border-black/15 px-3 py-1 hover:border-green">{t("bulk.invert")}</button>
            </div>

            <div className="mt-5">
              <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
            </div>

            <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
          </section>

          <SelectionFooter
            count={sel.count}
            totalSec={sel.totalSec}
            reelName={reelName}
            onReelNameChange={setReelName}
            onBuild={onBuild}
            building={build.status === "building"}
            warning={warning}
          />
        </main>
      );
    }

    function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
      if (load.status === "demo") {
        return (
          <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
            <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
          </p>
        );
      }
      if (load.status === "loading") {
        return (
          <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
```

```
283             <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
}} />
284           </p>
285         );
286       }
287       if (load.status === "error") {
288         return (
289           <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
290             <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
291           </p>
292         );
293       }
294       return (
295         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
296           <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
297         </p>
298       );
299     }
300
```

5. user (2026-06-21T14:33:57.488Z)

```
1       // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2       // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3       // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4       import { ApiError } from "../api/client";
5
6       /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7       export function parseDetail(body: string): string {
8         try {
9           const j = JSON.parse(body) as { detail?: unknown };
10          if (j && typeof j.detail === "string") return j.detail;
11        } catch {
12          /* not JSON ? fall through to the raw body */
13        }
14        return body;
15      }
16
17      /** Stable, localizable failure categories surfaced by the ingest flow.
18       *  `engine` carries the engine's own message in `detail`; the rest are localized by
code. */
19      export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
| "engine";
20
21      export interface IngestError {
22        code: IngestErrorCode;
23        /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
translated. */
24        detail?: string;
25      }
26
27      /** Classify a thrown client error into a stable code. A network throw (ApiError with no
`status`)
28       *  is the common "engine unreachable" case ? its own friendly code; any HTTP error
surfaces the
29       *  engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
30      export function classifyError(err: unknown): IngestError {
31        if (err instanceof ApiError) {
```

```
32          if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
engine truly unreachable
33          if (err.body) {
34            const detail = parseDetail(err.body);
35            if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
400 unsupported-scheme)
36          }
37          // A bare 5xx with no body is almost always the engine being down / a gateway that
can't reach it
38          // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
it running?"
39          // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
side mistake).
40          if (err.status >= 500) return { code: "networkError" };
41          return { code: "engine", detail: `HTTP ${err.status}` };
42        }
43        return { code: "engine", detail: String(err) };
44      }
45
```

6. assistant (2026-06-21T14:33:59.639Z)

Now let me read the components and the remaining files.

7. user (2026-06-21T14:34:00.588Z)

```
1       // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,
2       // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3       // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4       // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.
5
6       import { useTranslation } from "react-i18next";
7
8       export type BuildState =
9         | { status: "idle" }
10        | { status: "building" }
11        | { status: "done"; url: string; filename: string }
12        | { status: "error"; error: string };
13
14      interface Props {
15        build: BuildState;
16        onDismiss: () => void;
17      }
18
19      export function CompileResult({ build, onDismiss }: Props) {
20        const { t } = useTranslation();
21        if (build.status === "idle" || build.status === "building") return null;
22
23        return (
24          <section
25            data-testid="compile-result"
26            className={
27              "mt-6 rounded-xl border p-4 " +
28              (build.status === "error" ? "border-red-200 bg-red-50" : "border-green/30 bg-
green/5")
29            }
30          >
31            <div className="flex items-start justify-between gap-3">
32              <h2 className="font-semibold text-ink">
33                {build.status === "done" ? t("result.ready") : t("result.failed")}
34              </h2>
```

```
35              <button
36                type="button"
37                onClick={onDismiss}
38                aria-label={t("result.dismiss")}
39                className="rounded-full px-2 text-ink/50 hover:text-ink"
40              >
41                ?
42              </button>
43            </div>
44
45            {build.status === "done" ? (
46              <div className="mt-3">
47                {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
48                <video
49                  data-testid="reel-video"
50                  src={build.url}
51                  controls
52                  playsInline
53                  className="w-full max-w-xl rounded-lg bg-black"
54                />
55                <a
56                  href={build.url}
57                  download={build.filename}
58                  className="mt-3 inline-block rounded-full bg-green px-4 py-2 text-sm font-
semibold text-[#04140D]"
59                >
60                  {t("result.download", { filename: build.filename })}
61                </a>
62              </div>
63            ) : (
64              <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
65                {build.error}
66              </p>
67            )}
68          </section>
69        );
70      }
71
```

8. user (2026-06-21T14:34:01.042Z)

```
1       // The front-half ingest screen (appliance S1/S2, tracker B-P2). Point the tool at a
video ? a
2       // `gs://` bucket URI or a path on the engine host ? and it submits to POST
/api/process, shows
3       // live progress while polling GET /api/jobs/{id}, and hands the resulting video_id to
the existing
4       // rally-selection back-half (App lifts ?video=<id>). Honest by construction: the engine
reports
5       // coarse phase WORDS (not a %), and a success-shaped failure (done but no rallies) is
shown, never
6       // silently treated as a load.
7       import { useState } from "react";
8       import type { FormEvent } from "react";
9       import { useTranslation } from "react-i18next";
10      import { useIngestJob } from "../hooks/useIngestJob";
11
12      interface Props {
13        /** Called with the engine's video_id once a submitted job finishes successfully. */
14        onLoaded: (videoId: string) => void;
15      }
16
17      export function IngestPanel({ onLoaded }: Props) {
18        const { t } = useTranslation();
```

```
19        const [uri, setUri] = useState("");
20        const { phase, submit, cancel, reset, busy } = useIngestJob(onLoaded);
21
22        const onSubmit = (e: FormEvent) => {
23          e.preventDefault();
24          submit(uri);
25        };
26
27        // `engine` failures carry the engine's own message (a 400 scheme/path detail, a
worker error);
28        // every other code maps to a friendly localized line ? the network case especially,
since this
29        // is the first screen a fresh user hits and "is the engine running?" must read
plainly.
30        const errorMsg =
31          phase.status === "error"
32            ? phase.error.code === "engine"
33              ? (phase.error.detail ?? t("ingest.failed", { error: "" }))
34              : t(`ingest.${phase.error.code}`)
35            : null;
36
37        return (
38          <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
39            <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
40            <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
41
42            <form onSubmit={onSubmit} className="mt-3 flex flex-wrap items-center gap-2">
43              <input
44                type="text"
45                value={uri}
46                onChange={(e) => setUri(e.target.value)}
47                placeholder={t("ingest.uriPlaceholder")}
48                aria-label={t("ingest.uriAriaLabel")}
49                disabled={busy}
50                className="min-w-[18rem] flex-1 rounded-xl border border-black/15 bg-white
px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
51              />
52              <button
53                type="submit"
54                disabled={busy}
55                className="rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white
disabled:opacity-40"
56              >
57                {phase.status === "submitting" ? t("ingest.submitting") : t("ingest.submit")}
58              </button>
59              {busy && (
60                <button
61                  type="button"
62                  onClick={cancel}
63                  className="rounded-xl border border-black/15 px-4 py-2 text-sm hover:border-
green"
64                >
65                  {t("ingest.cancel")}
66                </button>
67              )}
68            </form>
69
70            <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
71
72            {/* Live status (aria-live so screen readers hear progress). Indeterminate: the
engine reports
73                phase words ("hashing", "segmenting"), not a percentage ? so a spinner + the
phase word. */}
74            <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
```

```
75              {busy && (
76                <p className="flex items-center gap-2 text-ink/70">
77                  <Spinner />
78                  <span>
79                    {t("ingest.working")}
80                    {phase.status === "polling" && phase.progress
81                      ? ` · ${t("ingest.progress", { progress: phase.progress })}`
82                      : ""}
83                  </span>
84                </p>
85              )}
86              {phase.status === "error" && (
87                <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
88                  <p>{errorMsg}</p>
89                  <button
90                    type="button"
91                    onClick={reset}
92                    className="mt-2 rounded-lg border border-red-300 px-3 py-1 text-xs
hover:bg-red-100"
93                  >
94                    {t("ingest.retry")}
95                  </button>
96                </div>
97              )}
98            </div>
99          </section>
100       );
101     }
102
103     function Spinner() {
104       return (
105         <span
106           className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
107           aria-hidden="true"
108         />
109       );
110     }
111
```

9. user (2026-06-21T14:34:01.546Z)

```
1       // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2       // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3       // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4       // text was understood, and concepts with no data behind them (shot-type / player /
score) are
5       // greyed out as "not available yet" ? never silently faked.
6       import { useMemo, useState } from "react";
7       import type { FormEvent } from "react";
8       import { useTranslation, Trans } from "react-i18next";
9       import { parseQuery, summarize } from "../lib/query";
10      import type { ParsedQuery } from "../lib/query";
11
12      interface Props {
13        /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
selection).
14         * `source` distinguishes a typed-and-submitted query from an example-chip tap (for
the trace). */
15        onApply: (q: ParsedQuery, source: "typed" | "example") => void;
16      }
```

```

    // Each example is a real, parseable query ? tapping one fills the box AND applies it. Led by rally
    // LENGTH (the trusted axis); shot-count filtering exists but is experimental, so it's not featured
    // here (a test pins that every example parses to a recognized, fully-available query).
    export const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "first 10"];

    const CHIP = "rounded-full px-2.5 py-0.5 text-xs font-medium";

    export function QueryBar({ onApply }: Props) {
      const { t } = useTranslation();
      const [value, setValue] = useState("");
      const parsed = useMemo(() => parseQuery(value), [value]);
      const clauses = summarize(parsed);
      const typed = value.trim().length > 0;

      const applyText = (text: string) => {
        setValue(text);
        onApply(parseQuery(text), "example");
      };
      const submit = (e: FormEvent) => {
        e.preventDefault();
        if (parsed.recognized) onApply(parsed, "typed");
      };

      return (
        <div>
          <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
            <input
              type="text"
              value={value}
              onChange={(e) => setValue(e.target.value)}
              placeholder={t("query.placeholder")}
              aria-label={t("query.ariaLabel")}
              className="min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green"
            />
            <button
              type="submit"
              disabled={!parsed.recognized}
              className="rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white disabled:opacity-40"
            >
              {t("query.apply")}
            </button>
          </form>

          {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing). */}
          <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap items-center gap-1.5">
            {clauses.map((c) => (
              <span key={c} className={CHIP + " bg-green/15 text-green"}>
                {c}
              </span>
            ))}
            {parsed.unavailable.map((u) => (
              <span
                key={u.token}
                title={t("query.notAvailableTitle", { token: u.token, kind: u.kind })}
                className={CHIP + " bg-black/5 text-ink/45"}
              >
                <span className="line-through">{u.token}</span> · {t("query.notAvailable")}
              </span>
```

```
76              ))}
77              {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
78                <span className="text-xs text-ink/50">{t("query.unreadable")}</span>
79              )}
80              {!typed && <span className="text-xs text-ink/50">{t("query.hint")}</span>}
81            </div>
82
83            <div className="mt-3 flex flex-wrap gap-2 text-sm">
84              <span className="text-ink/40">{t("query.try")}</span>
85              {EXAMPLES.map((ex) => (
86                <button
87                  key={ex}
88                  type="button"
89                  onClick={() => applyText(ex)}
90                  className="rounded-full border border-black/15 px-3 py-1 text-xs
hover:border-green"
91                >
92                  {ex}
93                </button>
94              ))}
95            </div>
96
97            <p className="mt-2 text-xs text-ink/40">
98              <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60"
/>, x: <span className="text-amber-700" /> }} />
99            </p>
100         </div>
101       );
102     }
103
```

10. user (2026-06-21T14:34:04.955Z)

```
1       // The selectable rally grid ? one card per play_segment, HONEST fields only:
2       // ordinal (by start order), mm:ss start, and rally LENGTH (the trusted axis). A shot-
count chip
3       // is shown ONLY when present, and clearly flagged EXPERIMENTAL ? shot detection is
still under
4       // development and not yet reliable, so length is what we lead with (owner 2026-06-20).
5       // Never renders server/receiver/events (motion-only fabrications,
PER_RALLY_SELECTION.md §4).
6       import { useTranslation } from "react-i18next";
7       import type { RallySegment } from "../api/types";
8       import { mmss } from "../lib/format";
9
10      interface Props {
11        rallies: RallySegment[];
12        isSelected: (id: number) => boolean;
13        onToggle: (id: number) => void;
14        /** Stable chronological label for a rally (by start_time), so re-sorting via a query
15         * doesn't renumber the cards. Falls back to display position when absent. */
16        ordinalOf?: (id: number) => number;
17      }
18
19      export function RallyGrid({ rallies, isSelected, onToggle, ordinalOf }: Props) {
20        const { t } = useTranslation();
21        if (rallies.length === 0) {
22          return <p className="text-ink/60 text-sm">{t("grid.empty")}</p>;
23        }
24        return (
25          <ul className="grid grid-cols-[repeat(auto-fill,minmax(150px,1fr))] gap-3 list-none
p-0 m-0">
26            {rallies.map((r, i) => {
27              const selected = isSelected(r.id);
28              const n = ordinalOf ? ordinalOf(r.id) : i + 1;
```

```
29              const shot = r.metadata?.shot_count;
30              const conf = r.metadata?.shot_count_confidence;
31              return (
32                <li key={r.id}>
33                  <button
34                    type="button"
35                    onClick={() => onToggle(r.id)}
36                    aria-pressed={selected}
37                    aria-label={t(selected ? "grid.inReel" : "grid.notInReel", { n })}
38                    className={
39                      "w-full text-left rounded-xl border p-3 transition " +
40                      (selected
41                        ? "border-green bg-green/10"
42                        : "border-black/10 bg-white hover:border-green/60")
43                    }
44                  >
45                    <div className="flex items-center justify-between">
46                      <span className="font-semibold text-ink">{t("grid.rally", { n })}</span>
47                      <span
48                        aria-hidden="true"
49                        className={
50                          "inline-flex h-5 w-5 items-center justify-center rounded-full text-
xs " +
51                          (selected ? "bg-green text-[#04140D]" : "border border-black/20
text-transparent")
52                        }
53                      >
54                        ?
55                      </span>
56                    </div>
57                    <div className="mt-1 text-sm text-ink/70">
58                      {mmss(r.start_time)} <span className="text-
ink/40">{t("grid.start")}</span> · <strong className="text-ink/90">{mmss(r.duration)}</strong>
<span className="text-ink/40">{t("grid.long")}</span>
59                    </div>
60                    {typeof shot === "number" && (
61                      <div
62                        title={t("grid.shotsTitle", {
63                          conf: conf && conf !== "Unknown" ? t("grid.shotsConfidence", { conf
}) : "",
64                        })}
65                        className="mt-2 inline-block rounded-full bg-amber-50 px-2 py-0.5
text-xs text-amber-700"
66                      >
67                        {t("grid.shots", { count: shot })}
68                      </div>
69                    )}
70                  </button>
71                </li>
72              );
73            })}
74          </ul>
75        );
76      }
77
```

11. user (2026-06-21T14:34:05.412Z)

```
1       // Sticky compile footer (PER_RALLY_SELECTION.md P4): live selection summary, a reel-
name input,
2       // the Build button, and ? above all ? an HONEST min_shot_count warning. The engine
silently drops
3       // selected rallies below stitching.min_shot_count; we surface that BEFORE Build so a
rally never
4       // vanishes without a reason (D7). We warn, we do NOT override the floor (D11).
```

```
      import { useTranslation, Trans } from "react-i18next";
      import { mmss } from "../lib/format";
      import type { FloorRisk } from "../lib/floor";

      interface Props {
        count: number;
        totalSec: number;
        reelName: string;
        onReelNameChange: (v: string) => void;
        onBuild: () => void;
        /** A compile is in flight (Build wired in P5). */
        building?: boolean;
        /** Non-null when some selected rallies sit below the engine floor. */
        warning: FloorRisk | null;
      }

      export function SelectionFooter({ count, totalSec, reelName, onReelNameChange, onBuild,
building, warning }: Props) {
        const { t } = useTranslation();
        const buildDisabled = count === 0 || building || warning?.allDropped === true;

        return (
          <footer className="fixed inset-x-0 bottom-0 border-t border-black/10 bg-white/95
backdrop-blur">
            {warning && (
              <div
                role="status"
                data-testid="floor-warning"
                className={
                  "border-b px-6 py-2 text-sm " +
                  (warning.allDropped
                    ? "border-red-200 bg-red-50 text-red-800"
                    : "border-amber-200 bg-amber-50 text-amber-900")
                }
              >
                {warning.allDropped ? (
                  count === 1 ? (
                    <Trans i18nKey="footer.floorOne" values={{ min: warning.minShotCount }}
components={{ b: <strong /> }} />
                  ) : (
                    <Trans i18nKey="footer.floorAll" values={{ count, min:
warning.minShotCount }} components={{ b: <strong /> }} />
                  )
                ) : (
                  <>
                    <strong>{t("footer.floorSomeLead", { count: warning.atRisk.length
})}</strong>{" "}
                    <Trans i18nKey="footer.floorSomeRest" values={{ min: warning.minShotCount
}} components={{ code: <code /> }} />
                  </>
                )}
              </div>
            )}

            <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-3 px-6 py-3">
              <span className="text-sm text-ink/80" data-testid="selection-summary">
                <strong className="text-ink">{t("footer.count", { count })}</strong>
{t("footer.summaryRest", { total: mmss(totalSec) })}
              </span>

              <label className="ml-auto flex items-center gap-2 text-sm text-ink/60">
                <span className="hidden sm:inline">{t("footer.reelName")}</span>
                <input
                  type="text"
                  value={reelName}
```

```
63                  onChange={(e) => onReelNameChange(e.target.value)}
64                  placeholder={t("footer.reelNamePlaceholder")}
65                  aria-label={t("footer.reelName")}
66                  className="w-40 rounded-lg border border-black/15 bg-white px-3 py-1.5 text-
ink outline-none focus:border-green"
67                />
68              </label>
69
70              <button
71                type="button"
72                onClick={onBuild}
73                disabled={buildDisabled}
74                className="rounded-full bg-green px-5 py-2 text-sm font-semibold
text-[#04140D] disabled:opacity-40"
75              >
76                {building ? t("footer.building") : t("footer.build", { count })}
77              </button>
78            </div>
79          </footer>
80        );
81      }
82
```

12. user (2026-06-21T14:34:05.871Z)

```
1       import { useTranslation } from "react-i18next";
2       import { SUPPORTED_LOCALES, DEFAULT_LOCALE, type Locale } from "../i18n";
3
4       // Each locale's label is its own autonym, shown in its own script. Kannada renders
crisply once the
5       // bundled Noto Sans Kannada font lands (tracker B4); most devices already ship a system
Kannada font.
6       const LANG_LABEL_KEY: Record<Locale, string> = {
7         en: "lang.en",
8         kn: "lang.kn",
9         hi: "lang.hi",
10      };
11
12      /**
13       * A persistent language switcher. Selecting a locale calls i18next's changeLanguage,
which the
14       * detector caches to localStorage (LANG_STORAGE_KEY) ? so the choice is the top-
priority signal on
15       * the next visit (negotiation: persisted pref wins). Icon-free + non-color active state
via
16       * aria-pressed keeps it usable for non-tech-savvy users (FRIENDLY_AND_MULTILINGUAL §4).
17       */
18      export function LanguageSwitcher() {
19        const { t, i18n } = useTranslation();
20        // Highlight the user's CHOSEN locale (i18n.language), not the resolved-for-content
one:
21        // resolvedLanguage falls back to `en` when a catalog is missing, so picking Kannada
(content
22        // still English until the kn catalog lands, B5) must still show Kannada as selected.
Compare on
23        // the base subtag so a detected "en-IN" highlights "en".
24        const active = (i18n.language || DEFAULT_LOCALE).split("-")[0];
25
26        return (
27          <nav aria-label={t("lang.label")} className="flex items-center gap-1">
28            {SUPPORTED_LOCALES.map((loc) => {
29              const isActive = active === loc;
30              return (
31                <button
32                  key={loc}
```

```
                  type="button"
                  lang={loc}
                  aria-pressed={isActive}
                  onClick={() => void i18n.changeLanguage(loc)}
                  className={[
                    "rounded-full px-3 py-1.5 text-sm font-semibold transition",
                    isActive ? "bg-white text-ink" : "text-white/70 hover:text-white",
                  ].join(" ")}
                >
                  {t(LANG_LABEL_KEY[loc])}
                </button>
              );
            })}
          </nav>
        );
      }

```

13. assistant (2026-06-21T14:34:08.225Z)

Now let me read the i18n catalog, hygiene test, index.css, and the docs section.

14. user (2026-06-21T14:34:09.638Z)

```
{
  "app": {
    "badge": "per-rally selection · alpha",
    "title": "Pick the rallies for your reel",
    "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
  },
  "lang": {
    "label": "Language",
    "en": "English",
    "kn": "?????",
    "hi": "??????"
  },
  "bulk": {
    "selectAll": "Select all",
    "clear": "Clear",
    "invert": "Invert"
  },
  "mode": {
    "demo": "Demo data ? open with <code>?video=?video_id?</code> against a running
engine to load a real session.",
    "loading": "Loading rallies for <code>{videoId}</code>?",
    "error": "Couldn't load rallies for <code>{videoId}</code>: {error}",
    "loaded": "Loaded <b>{count}</b> rallies from this session."
  },
  "query": {
    "placeholder": "e.g. over 10s · top 15 longest · shortest first",
    "ariaLabel": "Filter rallies by length, count or order",
    "apply": "Apply",
    "notAvailable": "not available yet",
    "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
    "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
    "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
    "try": "Sample queries:",
    "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
<x>experimental</x> (under development)."
  },
  "grid": {
    "empty": "No rallies to show yet.",
    "rally": "Rally {n}",
    "inReel": "Rally {n}, in reel",
```

```
39          "notInReel": "Rally {n}, not in reel",
40          "start": "start",
41          "long": "long",
42          "shots": "~{count} shots · experimental",
43          "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
44          "shotsConfidence": " ? confidence: {conf}"
45        },
46        "footer": {
47          "count": "{count, plural, one {# rally} other {# rallies}}",
48          "summaryRest": "selected · {total} total",
49          "reelName": "Reel name",
50          "reelNamePlaceholder": "my-highlights",
51          "build": "Build reel ({count})",
52          "building": "Building?",
53          "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? the engine
would skip it, so the build would fail. Pick a longer rally.",
54          "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ? the
engine skips every one, so the build would fail. Pick longer rallies.",
55          "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
56          "floorSomeRest": "with fewer than {min} shots will be skipped when building (the
engine's <code>min_shot_count</code> floor)."
57        },
58        "result": {
59          "ready": "Your reel is ready",
60          "failed": "Build failed",
61          "dismiss": "Dismiss",
62          "download": "Download {filename}"
63        },
64        "build": {
65          "noVideo": "Open with ?video=<video_id> against a running engine to build a reel ?
demo data can't be compiled."
66        },
67        "ingest": {
68          "title": "Process a video",
69          "intro": "Point KhelSutra at a video ? a bucket link or a file on the engine ? and
it finds the rallies. This can take a few minutes.",
70          "uriPlaceholder": "gs://your-bucket/match.mp4",
71          "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
72          "hint": "Paste a gs:// bucket link, or a video path on the machine running the
engine.",
73          "submit": "Process video",
74          "submitting": "Submitting?",
75          "cancel": "Cancel",
76          "working": "Processing your video?",
77          "progress": "{progress}",
78          "retry": "Try again",
79          "failed": "Couldn't process the video: {error}",
80          "invalidUri": "Enter a video location first ? a gs:// link or a path on the
engine.",
81          "networkError": "Can't reach the engine. Is it running?",
82          "emptyResult": "Processing finished but found no rallies in this video. Try a
clearer match recording.",
83          "timeout": "This is taking longer than expected. The job may still be running ?
reload with the video link, or try again."
84        },
85        "selection": {
86          "count": "{count, plural, one {# rally selected} other {# rallies selected}}"
87        }
88      }
89
```

15. user (2026-06-21T14:34:10.339Z)

```
1       // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
```

```
2       //   1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3       //      drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4       //      starts enforcing the moment a kn/hi catalog lands (B5/B6).
5       //   2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6       //      aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7       //      the doc; an AST-level sweep of JSX text can extend it later.
8       import { describe, it, expect } from "vitest";
9       import { readdirSync, readFileSync } from "node:fs";
10      import { join } from "node:path";
11
12      const LOCALES_DIR = join(import.meta.dirname, "locales");
13      const SRC_DIR = join(import.meta.dirname, "..");
14
15      function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
16        const keys: string[] = [];
17        for (const [k, v] of Object.entries(obj)) {
18          const key = prefix ? `${prefix}.${k}` : k;
19          if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
20          else keys.push(key);
21        }
22        return keys.sort();
23      }
24
25      const loadKeys = (locale: string): string[] =>
26        flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
27
28      describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
29        const enKeys = loadKeys("en");
30        const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
31          .filter((d) => d.isDirectory() && d.name !== "en")
32          .map((d) => d.name);
33
34        it("en is the non-empty source catalog", () => {
35          expect(enKeys.length).toBeGreaterThan(20);
36        });
37
38        for (const loc of others) {
39          it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
40            const keys = loadKeys(loc);
41            const missing = enKeys.filter((k) => !keys.includes(k));
42            const extra = keys.filter((k) => !enKeys.includes(k));
43            expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
44          });
45        }
46      });
47
48      describe("no-bare-strings ? user-facing component attributes go through t()", () => {
49        const FILES = [
50          "App.tsx",
51          "components/QueryBar.tsx",
52          "components/RallyGrid.tsx",
53          "components/SelectionFooter.tsx",
54          "components/CompileResult.tsx",
55          "components/LanguageSwitcher.tsx",
56          "components/IngestPanel.tsx",
57        ];
58        // A hardcoded placeholder / aria-label / title with letters is a bare-string
regression ? it
```

```
59        // must be an expression ({t(...)}), not a quoted literal.
60        const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"[^"]*[A-Za-z]{2}[^"]*"/g;
61
62        for (const f of FILES) {
63          it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
64            const src = readFileSync(join(SRC_DIR, f), "utf8");
65            expect(src.match(BARE_ATTR) ?? []).toEqual([]);
66          });
67        }
68      });
69
70      describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
71        it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google
Fonts)", () => {
72          const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
73          expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
74          expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
75        });
76        it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
77          const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
78          expect(css).toMatch(/Noto Sans Kannada Variable/);
79          expect(css).toMatch(/Noto Sans Devanagari Variable/);
80        });
81      });
82
83      // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
internet) ?
84      // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
CDN/Google Fonts.
85      describe("offline / self-contained ? no external runtime assets", () => {
86        const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr|\bcdn\.[a-z]/i;
87        const FILES = [
88          "../index.html", // web/index.html
89          "main.tsx",
90          "index.css",
91        ];
92        for (const rel of FILES) {
93          it(`${rel}: no external font/CDN reference`, () => {
94            expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
95          });
96        }
97        it("web/index.html loads no absolute-URL <link>/<script src> (everything is
bundled/local)", () => {
98          const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
99          expect(html).not.toMatch(/(?:href|src)\s*=\s*["']]https?:\/\//i);
100       });
101     });
102
```

16. user (2026-06-21T14:34:10.342Z)

```
1       @import "tailwindcss";
2
3       /* Brand tokens (mirror the marketing site palette). Tailwind v4 generates utilities
4          from these ? e.g. bg-paper, text-ink, bg-ink, text-lime, text-green. */
5       @theme {
6         --color-ink: #0c261f;
7         --color-ink2: #123a2e;
8         --color-green: #19b36b;
9         --color-lime: #c7f2b0;
10        --color-paper: #f6f8f4;
11      }
12
13      /* Bundled OFL Noto fonts (B4 / D7): Latin uses the system stack; Kannada & Devanagari
codepoints
```

```
14         resolve to the self-hosted Noto faces (unicode-range-scoped), so kn/hi render crisply
with no
15         Google Fonts runtime dependency. Order matters ? the Latin stack is matched first for
Latin. */
16       body {
17         font-family:
18           ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, Helvetica, Arial,
19           "Noto Sans Kannada Variable", "Noto Sans Devanagari Variable", sans-serif;
20       }
21
```

17. assistant (2026-06-21T14:34:13.635Z)

Now let me read the docs section §7 and §4 for A4/A5/A8 rows and UX levers.

18. user (2026-06-21T14:34:14.929Z)

```
1        # Friendly & Multilingual ? non-tech-savvy UX + Hindi/Kannada localization
2
3        > **Status:** `DRAFT` ? owner-gated (implementation starts only on explicit approval) ·
**Owner:** Avi Dullu · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
4        > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE).
5        > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; thread in `NEXT_STEPS.md`; pointer in engine `docs/DOC_STATUS.md`; memory
`current-state` + `next-session-ux-localization`.
6        > **Relation to existing docs:** this is the **implementation project** for two engine
design docs ? it *adapts & supersedes-in-practice* `badminton-highlight-
indexer/docs/I18N_PLAN.md` (localization) and the UX half of `?/docs/UI_PROPOSAL.md` (non-tech-
savvy console). Those stay the canonical *design*; this doc carries the *tracked execution*
against today's surfaces.
7        > **Honesty note:** each claim tagged `[verified]` (checked against code this session),
`[researched]` (needs sign-off), or `[design]` (proposed). Not legal advice ? the OFL font
question (D-FONT) is owner/counsel-gated.
8
9        ---
10
11       ## 0. TL;DR
12       A coach or parent in Bangalore should land on KhelSutra, feel at home, set up their
camera with confidence, and get highlights ? **in Kannada or Hindi**, with **zero hand-
holding**. This project isolates that goal into one tracked workstream-pair: **(A)** make the UI
super friendly for non-tech-savvy users, and **(B)** localize to Hindi + Kannada. Both surfaces
(`web/` React SPA + `site/` vanilla marketing) have **zero i18n today** `[verified]`, so the
foundation ? shared `locales/{en,kn,hi}` catalogs + a negotiation contract + CI parity ? is the
real deliverable; translations follow. **The single most important caveat:** translations are
**human-reviewed** (Gemini drafts are *inference only, never training* ? the standing
guardrail), and Kannada/Hindi rendering is **blocked on the OFL font decision (D-FONT)**. The
cheapest unblocked high-impact first slice is the **SPA i18n scaffold + language switcher (en-
only)** ? it locks the shared catalog/negotiation contract and adds the render-test safety net
the SPA currently lacks.
13
14       ## 1. Problem & goal
15       **Why now.** The public site (khelsutra.guru) is live, clean, honest, and tested. The
PMF beachhead is **Bangalore badminton academies**; many coaches, parents, and players are far
more comfortable in **Kannada** than English, and most are **not highly tech-savvy** `[verified
? design intent]` (`UI_PROPOSAL.md:25,28`, `I18N_PLAN.md:10-13`, `PMF_MARKET_RESEARCH.md:116`,
`KHELSUTRA_PRODUCT_OVERVIEW.md:13/50/61`). An English-only, knob-heavy product adds friction at
the exact point of first contact.
16
17       **Concrete outcome ("good" looks like):** a Kannada/Hindi-comfortable, phone-first, non-
technical coach/parent can ? without help ? (1) understand what the product does and how to
record reliably, (2) request access / use the console, (3) pick rallies and get a reel, and (4)
share it on WhatsApp, **entirely in their language**.
18
19       **Read to get current:** this doc's §3?§4; the engine design `I18N_PLAN.md` (canonical
```

i18n design) + `UI_PROPOSAL.md` §2 (personas, the 6 alpha screens, "teach recording *before* upload"); `VIDEO_RECORDING_GUIDELINES.md`; this repo's `PER_RALLY_SELECTION.md` (the honest SPA scope) + `REQUEST_ACCESS_FORM.md`.

2. Decisions locked (structural ? owner directive 2026-06-21)

#	Decision	Source / date	Implication
D1	**Isolate as ONE tracked project** spanning both workstreams (A: UX, B: kn/hi l10n)	owner, 2026-06-21	This doc; §7 = source of truth; archived on DoD
D2	**Home = `khelsutra/docs/`** (the implementation repo)	owner-gated tracked-doc convention	`web/` + `site/` both live here; §7 tracks khelsutra PRs; engine P2 backend-l10n is a cross-repo deferred row
D3	**Adapt `I18N_PLAN.md` to today's surfaces:** P1 (vanilla `t()` shim) ? `site/`; P3 (react-i18next) ? `web/`; **drop the dead `backend/api/static` retrofit**	`[verified]` the doc targets `backend/api/static` (`I18N_PLAN.md:25,30,156,191`) which exists only in the engine repo, not here	The doc's framework-agnostic *foundation* (catalog format + negotiation + ICU plurals + CI) survives unchanged; only the wiring re-points
D4	**Engine P2 backend-localization stays owner-gated + cross-repo** (not in this DoD)	`CLAUDE.md` guardrails	Dynamic engine error/report text localizes later; tracked as a deferred row
D5	**Archive trigger = en+kn+hi live on BOTH surfaces** (+ the non-tech UX baseline), then run the engine `DOC_STATUS.md §2` archival checklist	owner, 2026-06-21	See §9
D6	**D-LOCALES: `en` + `kn` for v1, `hi` next; always-fallback to `en`**	owner, 2026-06-21	Kannada-first beachhead; `hi` = catalog-only (both 2-form CLDR)
D7	**D-FONT: OFL-for-media carve-out ? self-host Noto Sans Kannada/Devanagari** (counsel may still confirm)	owner, 2026-06-21	Unblocks B4/B5/A3 + the kn/hi reel watermark; an explicit exception to the MIT/Apache/BSD code guardrail (fonts = media)
D8	**D-SURFACE-FIRST: lead with the `web/` SPA scaffold; SKIP the dead static-UI retrofit**	owner, 2026-06-21 (via B1 go-ahead)	B1 is greenlit; site recording-education localized early
D9	**D-CATALOG-HOME: co-locate `locales/` + CI in `khelsutra`** (shared, consumed by SPA + site)	owner, 2026-06-21 (via B1 go-ahead)	No cross-repo coupling

*Only **D-BACKEND-DEPTH** (§8.3) remains open ? and it's a deferred, cross-repo engine concern (B7), not a v1 blocker.*

3. Foundation ? research / prior art

Current-state facts `[verified]` this session (adversarially checked against the working tree):

- **`web/` SPA: zero i18n.** `web/package.json` deps are only `react`/`react-dom`; no `i18next`/`react-intl`/`formatjs`; no `locales/` dir; no `t()`/`useTranslation`/`data-i18n` anywhere. User-facing English is hardcoded in `.tsx` (e.g. `QueryBar.tsx:46-47,56,72,76,78`; `App.tsx:115,211,224`) ? ~50 literals + ~16 derived labels.
- **`site/` marketing: vanilla, zero i18n.** Three `<html lang="en">` files (`public/{index,capture,own-model}.html`) with inline CSS, no framework; the only scripts are a CF analytics beacon (`index.html:595`) + one inline form/reel IIFE (`index.html:597-633`). `server.js` hardcodes English responses; no locale handling.
- **Recording-education is *not* greenfield.** The homepage "Record it right" gist (`index.html:324-364`) + the `/capture` checklist (`capture.html`, routed `server.js:51`, guarded `assets.test.js:94`) already ship the user-facing best-practices surface. *Unshipped:* the empirical-threshold framing and the programmatic pre-flight validator (a code change, not a page).
- **Persona is design intent, not built.** kn/hi + zero-knobs are **proposed/owner-gated**; the app is English-only today (`KHELSUTRA_PRODUCT_OVERVIEW.md:61`). It's Kannada-**first**, Hindi-**next** ? not Kannada-only.

Localization prior art `[researched]`:

- **CLDR plural categories:** Hindi = `{one, other}`; Kannada = `{one, other}`. Both are 2-form (same shape as English) ? once ICU MessageFormat is in the catalog, adding `hi`/`kn` plurals is **catalog-only**, no code. (ICU still required because English `count>1?'s':''` hand-rolling ? present today in the SPA ? doesn't generalize.)
- **react-i18next** (web/): `i18next` + `react-i18next` + `i18next-browser-languagedetector` + an ICU/plural post-processor; lazy-load per-locale JSON; persist via `localStorage`.

```
46      - **Shared catalog + tiny shim** (site/): the SAME i18next-flavored JSON consumed by a
~30-line `t(key, vars)` shim that walks `data-i18n` attributes ? no build step, no framework,
mirrors i18next interpolation/plural rules for the 2-form languages.
47      - **Noto Sans Kannada / Noto Sans Devanagari** are **OFL-1.1**; OFL permits commercial
bundling/redistribution (fonts = media, not code/weights). Self-host (e.g. `@fontsource`) with
`font-display: swap` + subsetting. **Gated on D-FONT.**
48      - **Non-tech-savvy / HCI4D patterns:** icon **+ text** (never icon-only), non-color
selection indicators, large touch targets, mobile-first, WhatsApp as the share channel, a
prominent persistent language switcher.
49
50      ## 4. Design / architecture
51      **Two surfaces, one shared source of truth.**
52
53      ```
54      locales/                      # shared catalogs ? co-located in khelsutra (D-CATALOG-
HOME, see §8)
55        en/  common.json            # source of truth ? devs author keys here
56        kn/  common.json            # Kannada  (Gemini-draft ? human-review)
57        hi/  common.json            # Hindi    (next ? the extensibility proof)
58
59      web/   ? react-i18next        # I18N_PLAN P3: provider in main.tsx; useTranslation in
components
60      site/  ? t(key,vars) shim      # I18N_PLAN P1 retargeted: data-i18n attrs over the
static HTML
61      ```
62
63      - **Negotiation contract (one order, both surfaces):** `persisted pref (localStorage) ?
switcher / ?lang=<code> ? navigator/Accept-Language ? "en"`. Supported locales in a single
module; anything else falls back to `en`; **a missing key degrades to English, never to a raw
key.**
64      - **Keys, never prose** ? `t("compile.button")`, with ICU for interpolation/plurals:
`"{count, plural, one {# rally} other {# rallies}}"`. This replaces the SPA's current hand-
rolled `Segment${count>1?'s':''}`.
65      - **Translation pipeline (guardrail-clean):** dev authors `en` keys ?
`scripts/i18n_draft.py`-style Gemini **draft** (inference only, never training; flags every
value for review) ? **owner/native-speaker review** ? CI key-parity ? merge. Gemini never
touches `en`, never invents keys.
66      - **CI enforcement (keeps it "by design"):** (1) **key-parity** ? every locale has
exactly the `en` key set (drop a stub `ta.json` ? CI lists every gap = the extensibility proof);
(2) **no-bare-strings** ? flags user-facing literals not routed through `t()` (curated allowlist
to start).
67      - **Fonts:** bundle Noto Sans Kannada + Devanagari **iff D-FONT = OFL carve-out**; this
also unblocks a Kannada/Hindi **reel watermark** (engine `WatermarkConfig.font_path` already
takes a path ? config, not code).
68
69      **Non-tech-savvy UX levers** (ranked impact × effort, grounded in personas; surface-
tagged):
70
71      | Lever | Surface | Impact | Effort | Note |
72      |---|---|---|---|---|
73      | Plain-language copy ? strip engine jargon, keep honest caveats | both | high | small |
banners + footer carry backend vocabulary today |
74      | Localized **WhatsApp share** for the downloaded reel | both | high | small | WhatsApp
is *the* channel for this audience |
75      | **Language switcher** persisted via a shared localStorage key | both | high | small |
site promises kn/hi but ships no switcher |
76      | Localize **recording-education** (capture + gist) kn?hi | site | high | medium | gates
input quality; the highest-leverage kn content |
77      | Mobile-first touch targets; tooltip-text ? visible | web | high | medium | tiny
targets + desktop layout + invisible tooltips on phones |
78      | Icon **+ text** + non-color selection indicator | web | medium | small | never icon-
only |
79      | Friendly engine-error mapping over raw FastAPI `detail` | web | medium | medium |
`CompileResult` passes engine text untranslated |
80      | Plain-language framing for the Request-Access GPU/storage Qs | site | medium | small |
```

"GPU"/"S3/GCS" are intimidating |
81 | Background-processing reassurance + status as icon chips | web | medium | small | honest expectations on slow uplinks |
82 | Fix English-source recording gaps (a diagram) before translating | site | medium | medium | the diagram is reusable across all 3 languages |
83 | Tap-only filter controls alongside the English query bar | web | medium | large | the `parseQuery()` parser is English-grammar only |
84 | Keep the honest *today-vs-roadmap* split through translation | both | medium | small | never soften "experimental"/"not available yet" |
85
86 **Reuse map (what's new vs reused):** *Reused* ? the i18next-flavored JSON + negotiation contract + ICU plurals + key-parity/no-bare-strings CI + draft?review pipeline (all from `I18N_PLAN.md`); the shipped recording pages; the SPA's `lib/format.ts`/`lib/query.ts`; `site/src/assets.test.js` + the SPA's vitest harness. *New* ? `locales/` catalogs; the SPA react-i18next provider + switcher; the site `t()` shim + `data-i18n` wiring; Noto font bundle (gated); a jsdom render-test env for the SPA.
87
88 ## 5. Privacy / scope note
89 i18n touches **presentation only** ? no new personal-data collection, no biometrics, minors-excluded posture unchanged. WhatsApp share opens the user's own client with a pre-filled message; it sends nothing server-side. Gemini-draft translation is inference, not training (no user data in the loop).
90
91 ## 6. Honest limits ? what this does NOT do
92 - A green CI / DONE status proves **key-parity + render + no-bare-strings**, **not** translation *quality* ? that's human-gated and owner-paced.
93 - Kannada/Hindi will **not render** until the Noto fonts land (D-FONT); until then kn/hi catalogs exist but display tofu/fallback.
94 - The SPA is **job-tethered**; localizing **dynamic engine errors/report text** needs engine P2 (owner-gated, cross-repo, D-BACKEND-DEPTH) ? out of this DoD.
95 - The query bar stays **English-grammar parsing**; kn/hi users rely on tap-only controls (a later, larger slice), not free-text Kannada queries.
96 - **Mobile-*perfect*** layout is an alpha non-goal (usable-on-phone is in scope; pixel-perfect is not).
97
98 ## 6.5 Testing strategy ? automatic language negotiation (rigorous, owner-required)
99 **The bar (owner, #41):** a user whose **computer/browser language is Kannada or Hindi must get the respective version automatically** ? that is the core promise and it is **proven by tests at every layer, never assumed.** Auto-selection IS the negotiation order (persisted pref ? `?lang=` ? browser/`Accept-Language` ? `en`); each step gets a test.
100
101 **Negotiation matrix ? every row is an automated test:**
102
103 | # | Signal under test | Setup | Expected | Proven in |
104 |---|---|---|---|---|
105 | N1 | **Browser/OS = Kannada** | no persisted pref, no `?lang`, `navigator.languages=["kn-IN", ?]` | resolves to **kn** ? Kannada content (once the kn catalog ships) | **B1** detection · **B5** content |
106 | N2 | **Browser/OS = Hindi** | `navigator.languages=["hi-IN", ?]` | resolves to **hi** ? Hindi content | **B6** (hi added to the supported set + catalog) |
107 | N3 | **`?lang=` override** | `?lang=kn`, browser = en | kn | B1 |
108 | N4 | **Persisted pref wins** | `localStorage=kn`, browser = en, no `?lang` | kn (a returning user keeps their choice) | B1 |
109 | N5 | **Region subtag normalizes** | `navigator.language="kn-IN"` | base **kn** (not a "kn-IN" miss) | B1 |
110 | N6 | **Unsupported language** | `navigator="ta-IN"` (Tamil, not yet supported) | **en** fallback ? never a locale we can't serve | B1 |
111 | N7 | **Missing key in active locale** | kn active, a key absent from `kn/common.json` | **en** fallback for that key (no raw key shown) | B1/B5 |
112 | N8 | **Static site (vanilla shim)** | `Accept-Language: kn` / `?lang=kn` on a `site/` page | kn-rendered page (same order as the SPA) | B3 |
113 | N9 | **Backend (engine P2)** | `Accept-Language: kn` to the API | kn customer/report text | B7 (owner-gated) |
114
115 **Levels & tooling:**

- **SPA (`web/`):** jsdom tests stub `navigator.language`/`navigator.languages`, `localStorage`, and the `?lang` query ? assert `i18n.language`/`resolvedLanguage` + rendered text per the matrix. *(B1 lands N1/N3?N7 at the **detection** level now ? `kn` is supported; the kn/hi **content** assertions ride the catalogs in B5/B6.)*
- **Static site (B3):** the shared `t()` shim's negotiation is unit-tested the same way; plus a server test that `Accept-Language: kn`/`?lang=kn` returns the kn-rendered page.
- **CI key-parity:** guarantees every locale has every key, so a correctly-negotiated locale can never fall through to a raw key.
- **End-to-end (manual, owner machine ? per I18N_PLAN §6):** set the real OS/browser to Kannada, then Hindi ? load both surfaces ? confirm the right language renders **and the Noto fonts display** (the cloud CI container can't render fonts; the owner eyeball is the final gate).
- **? Query parsing / semantic understanding (forward-looking, owner directive 2026-06-21) ? i18n testing is MANDATORY there.** The query bar is **English-grammar-only** today (a documented limit, tracker A9): the user types English queries (`over 10s`, `top 15 longest`) and a localized **sample-query prompt** shows them what to type in *every* locale (kn/hi included). The **moment** the parser gains **semantic understanding** or accepts **Kannada/Hindi query input** (e.g. typing a Kannada/Hindi phrase, or a conversational/NL builder ? I18N_PLAN P4), i18n tests become a **merge gate**: a kn/hi query (typed or spoken) must parse to the **correct selection**, the localized **parse-feedback** ("parsed-pill") must render in the active locale, and the unsupported/empty-result messages must localize. **No query-NL / semantic-parsing work merges without those tests.**

A locale that's reachable by the switcher but **not auto-selected from the browser language is a bug, not a "later".** This matrix is a DoD gate (§9).

7. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking.

ID	Deliverable	Workstream	Depends on	Gated?	Status	PR
P0	This tracked project doc + inbound pointers	?	?	No	?	[#41](https://github.com/avidullu/khelsutra/pull/41) · engine [#275](https://github.com/Khelsutra/badminton-highlight-indexer/pull/275)
B1	**SPA i18n scaffold + language switcher, en-only** (react-i18next + detector + `en` catalog + switcher + persistence + **jsdom render tests** + **browser-language negotiation tests** §6.5 N1/N3?N7)	B	P0	No	?	[#42](https://github.com/avidullu/khelsutra/pull/42)
B2	Extract SPA strings ? `en` catalog (~50 literals + ~16 derived; ICU plural keys); add **key-parity + no-bare-strings CI**	B	B1	No	?	[#45](https://github.com/avidullu/khelsutra/pull/45)
A1	Plain-language copy pass on the `en` catalog (de-jargon; icon+text; non-color selection)	A	B2	No	?	?
B3	Tiny shared `t()` shim + `data-i18n` plumbing for `site/`, en-only (proves shared catalogs across surfaces)	B	B2	D-CATALOG-HOME	?	?
A2	Localized **WhatsApp share** for the reel (SPA + site)	A	B1/B3	No	?	?
A6	Plain-language framing for Request-Access GPU/storage questions	A	B3	No	?	?
A7	Fix English-source recording gaps (add a placement diagram) before localizing	A	?	No	?	?
B4	**Noto Sans Kannada + Devanagari** bundle (OFL carve-out) ? *SPA done; reel watermark `font_path` = engine follow-up*	B	?	D-FONT ?	?	[#46](https://github.com/avidullu/khelsutra/pull/46)
B5	**Kannada (`kn`) catalog** ? render ? *SPA done (AI-draft, native review pending); `site/` rides B3*	B	B2/B4	D-LOCALES ?	?	[#47](https://github.com/avidullu/khelsutra/pull/47)
A3	Localize **recording-education** (capture + homepage gist) to kn	A	B3/B5	D-LOCALES+D-FONT	?	?
A4	Mobile-first touch targets + tooltip-text ? visible (SPA)	A	B1	No	?	?
A5	Friendly engine-error mapping over raw FastAPI `detail` (SPA, client-side map)	A	B2	No	?	?
B6	**Hindi (`hi`) catalog** ? render (the extensibility proof ? just a catalog) ? *SPA done (AI-draft, native review pending)*	B	B5	D-LOCALES ?	?	[#48](https://github.com/avidullu/khelsutra/pull/48)

A8	Background-processing reassurance + verdict/status as icon chips (SPA)	A	B2	No	?	?
A9	Tap-only filter controls alongside the English query bar (SPA)	A	B2	No	?	?
B7	*(deferred, cross-repo)* Engine **P2 backend customer-facing localization** (report `customer_*` + rejection reasons)	B	engine	**D-BACKEND-DEPTH, owner-gated**	?	?

? MILESTONE ? SPA fully trilingual (2026-06-21): the `web/` Studio SPA now speaks **en + kn + hi** end to end (B1?B2?B4?B5?B6, all merged). A **Kannada-default browser auto-renders Kannada**, and the switcher walks kn ? hi ? en ? **proven by a Playwright screencast** with `locale: kn-IN` (`khelsutra-i18n-screencast.mp4`; frames captured per locale + `<html lang>` tracked). This meets the owner's success criterion **on the SPA surface**. **Still open for the full §9 DoD:** **B3** (the `site/` marketing surface via the shared `t()` shim ? the homepage/capture pages are still English-only), **A3** (recording-education in kn/hi), the **non-tech UX A-items**, and a **native-speaker review** of the AI-drafted kn/hi catalogs before a wide public launch. Do **not** archive until both surfaces are trilingual.

8. Open questions ? owner (blocking gates) `[design]`

*Status (2026-06-21): 4 of 5 **LOCKED/ADOPTED** by the owner ? see §2 (D6?D9). Only D-BACKEND-DEPTH remains open (and it's deferred, not a v1 blocker).*

1. **D-FONT ? ? LOCKED ? (a) OFL-for-media carve-out, self-host Noto Sans Kannada/Devanagari** (counsel may still confirm). An explicit exception to the MIT/Apache/BSD code guardrail (fonts = media). Unblocks B4/B5/A3 + the kn/hi reel watermark.
2. **D-LOCALES ? ? LOCKED ? `en` + `kn` for v1, `hi` next, always-fallback to `en`** (both 2-form CLDR ? `hi` is catalog-only).
3. **D-BACKEND-DEPTH ? ? OPEN (deferred, not a v1 blocker).** Recommend **customer verdict/report `customer_*` + validation-rejection reasons first; technical/operator text stays English; phase two** (engine repo, owner-gated ? tracker row B7).
4. **D-SURFACE-FIRST ? ? ADOPTED ? lead with the SPA scaffold (B1), localize site recording-education early, SKIP the dead static-UI retrofit.**
5. **D-CATALOG-HOME ? ? ADOPTED ? co-locate `locales/` + CI in `khelsutra`** (a shared i18n dir consumed by both the SPA and the site). (The engine `I18N_PLAN` assumed the indexer repo; this is the adaptation.)

9. Definition of done

- [ ] **Automatic language negotiation (§6.5) is green** ? a **Kannada browser gets Kannada and a Hindi browser gets Hindi** (N1/N2); persisted-pref / `?lang` / region-subtag / unsupported-fallback / missing-key-fallback all proven (N3?N7) on the SPA, and the static-site shim honors `Accept-Language` + `?lang` (N8).
- [ ] **Manual cross-check (owner machine):** OS/browser set to Kannada and to Hindi each render the respective language on BOTH surfaces, with Noto fonts displaying.
- [ ] `en + kn + hi` catalogs live and **rendering** on BOTH `web/` SPA and `site/` marketing (switcher + persistence + `en` fallback; no raw keys shown).
- [ ] **Noto Sans Kannada + Devanagari render correctly** ? owner eyeball on a real machine.
- [ ] CI **key-parity + no-bare-strings** green; a stub locale flags every gap.
- [ ] **Recording-education** (capture + homepage gist) available in `kn` + `hi`.
- [ ] **Non-tech UX baseline:** plain-language copy (no raw engine jargon in user-facing strings), mobile-usable touch targets, icon+text, WhatsApp share, honest today-vs-roadmap preserved through translation.
- [ ] §7 all rows ? (or explicitly deferred with a reason) ? flip **Status ? DONE** + completion note ? run engine `DOC_STATUS.md §2` archival checklist ? `git mv` to `docs/archives/`.

10. References

Internal code anchors `[verified]`: `web/package.json`, `web/src/{App.tsx,main.tsx,components/QueryBar.tsx,components/CompileResult.tsx,lib/query.ts}`; `site/public/{index,capture,own-model}.html`, `site/server.js`, `site/src/assets.test.js`. **This repo's docs:** `docs/PER_RALLY_SELECTION.md`, `docs/REQUEST_ACCESS_FORM.md`, `docs/README.md`, `NEXT_STEPS.md`. **Engine design docs (canonical):** `badminton-highlight-indexer/docs/{I18N_PLAN.md,UI_PROPOSAL.md,VIDEO_RECORDING_GUIDELINES.md,PMF_MARKET_RESEARCH.md,PMF_FIELD_SIGNALS.md,KHELSUTRA_PRODUCT_OVERVIEW.md,DOC_STATUS.md}`. **External `[researched]`:** Unicode CLDR plural rules (kn/hi = one/other); react-i18next + i18next-icu; SIL Open Font License 1.1; Google Noto Sans Kannada/Devanagari.

```
169
170     ---
171
172     ### Changelog
173     - `2026-06-21` ? created. Isolated the UX + Hindi/Kannada work into one tracked project
(owner directive). Grounded in a multi-agent research+verification pass (web/+site/ string maps,
Indic-l10n prior art, 5 adversarial verdicts). Decisions D1?D5 locked; 5 product/licensing
decisions surfaced as open; §7 tracker seeded with B1 as the recommended first slice.
174     - `2026-06-21` ? owner locked **D6 (en+kn?hi)** + **D7 (OFL Noto carve-out)** and
adopted **D8 (SPA-first)** + **D9 (catalogs in khelsutra)**; only D-BACKEND-DEPTH remains open.
**B1 greenlit ? ? in progress.**
175     - `2026-06-21` ? **B1 opened ? [#42](https://github.com/avidullu/khelsutra/pull/42)**:
SPA i18n scaffold (react-i18next + ICU + detector) + `en` catalog + language switcher + the
SPA's first jsdom render tests. Verified typecheck/79-tests/build green + live in-browser
(switcher persists, en-fallback). Doc/pointer PRs opened: khelsutra
[#41](https://github.com/avidullu/khelsutra/pull/41), engine
[#275](https://github.com/Khelsutra/badminton-highlight-indexer/pull/275).
176     - `2026-06-21` ? **§6.5 Testing strategy added (owner ask, #41):** a rigorous
**automatic-negotiation matrix** (N1?N9) so a Kannada/Hindi **browser** auto-gets the respective
version ? proven at the detection level now (B1), at the content level with the catalogs
(B5/B6), on the static shim (B3), and by a manual font-render eyeball. Wired into the DoD (§9).
B1 (#42) gains the detection tests (N1/N3?N7).
177     - `2026-06-21` ? **? SPA MILESTONE: fully trilingual (owner directive "take it to
completion + screencast").** B2 (#45, full string extraction + key-parity/no-bare-strings CI) ?
B4 (#46, self-host OFL Noto) ? B5 (#47, Kannada) ? B6 (#48, Hindi ? the extensibility proof) all
merged. 99 web tests green incl. N1/N2 browser-detection + B5/B6 render + key-parity.
**Playwright screencast** (`locale: kn-IN`): a Kannada-default browser auto-renders Kannada ?
switch hi ? switch en, each with `<html lang>` tracked (`khelsutra-i18n-screencast.mp4` + per-
locale frames). kn/hi are **AI-drafted ? native review pending**. **Remaining for full §9 DoD:**
B3 (the `site/` marketing surface), A3 + the UX A-items, native-speaker translation review.
178
```

19. assistant (2026-06-21T14:34:19.612Z)

I have all the context I need. Let me check the kn/hi catalogs exist (for key-parity requirement) and look at the useIngestJob hook to understand the build/ingest error shape fully.

20. user (2026-06-21T14:34:21.105Z)

```
total 4
drwxr-xr-x 1 avidu 197609 0 Jun 21 12:43 .
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 ..
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 en
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 hi
drwxr-xr-x 1 avidu 197609 0 Jun 21 15:45 kn
---KN keys top---
hi:
common.json

kn:
common.json
```

21. user (2026-06-21T14:34:21.418Z)

```
1       // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2       // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3       //
4       // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5       // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6       // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
```

```
slow engine
7       // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8       // cancellable on unmount / user-cancel / a superseding submit.
9       import { useCallback, useEffect, useRef, useState } from "react";
10      import { getJob, processVideo } from "../api/client";
11      import { classifyError, type IngestError } from "../lib/errors";
12      import { log, newCorrelationId } from "../lib/log";
13      import type { JobStatus } from "../api/types";
14
15      const POLL_MIN_MS = 1000;
16      const POLL_MAX_MS = 5000;
17      const POLL_BACKOFF = 1.5;
18      const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20      export type IngestPhase =
21        | { status: "idle" }
22        | { status: "submitting" }
23        | { status: "polling"; progress: string | null }
24        | { status: "done"; videoId: string }
25        | { status: "error"; error: IngestError };
26
27      interface Run {
28        cancelled: boolean;
29        timer?: ReturnType<typeof setTimeout>;
30      }
31
32      export function useIngestJob(onLoaded?: (videoId: string) => void) {
33        const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34        const runRef = useRef<Run | null>(null);
35        // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36        const onLoadedRef = useRef(onLoaded);
37        onLoadedRef.current = onLoaded;
38
39        const stop = useCallback(() => {
40          const run = runRef.current;
41          if (run) {
42            run.cancelled = true;
43            if (run.timer) clearTimeout(run.timer);
44          }
45          runRef.current = null;
46        }, []);
47
48        // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49        useEffect(() => stop, [stop]);
50
51        const submit = useCallback(
52          (rawUri: string) => {
53            const uri = rawUri.trim();
54            if (!uri) {
55              setPhase({ status: "error", error: { code: "invalidUri" } });
56              return;
57            }
58
59            stop(); // supersede any prior run
60            const run: Run = { cancelled: false };
61            runRef.current = run;
62
63            const cid = newCorrelationId();
64            const startedAt = Date.now();
65            setPhase({ status: "submitting" });
66            log("info", "ingest.submit_start", { cid, uri });
67
```

```
68            const fail = (error: IngestError, event: string) => {
69              if (run.cancelled) return;
70              setPhase({ status: "error", error });
71              const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72              log(level, event, { cid, uri, code: error.code, detail: error.detail });
73            };
74
75            const poll = (jobId: string, delay: number) => {
76              run.timer = setTimeout(() => {
77                void (async () => {
78                  if (run.cancelled) return;
79                  let job: JobStatus;
80                  try {
81                    job = await getJob(jobId, cid);
82                  } catch (err) {
83                    fail(classifyError(err), "ingest.poll_error");
84                    return;
85                  }
86                  if (run.cancelled) return;
87
88                  if (job.status === "done") {
89                    const videoId = job.video_id ?? job.result?.video_id ?? null;
90                    // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                    // a success-shaped FAILURE ? never load it, never hang.
92                    if (job.result?.status === "failed" || !videoId) {
93                      fail({ code: "emptyResult" }, "ingest.empty_result");
94                      return;
95                    }
96                    setPhase({ status: "done", videoId });
97                    log("info", "ingest.done", { cid, uri, video_id: videoId });
98                    onLoadedRef.current?.(videoId);
99                    return;
100                 }
101                 if (job.status === "failed") {
102                   fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103                   return;
104                 }
105                 // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106                 setPhase({ status: "polling", progress: job.progress ?? null });
107                 if (Date.now() - startedAt > POLL_CEILING_MS) {
108                   fail({ code: "timeout" }, "ingest.poll_timeout");
109                   return;
110                 }
111                 poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112               })();
113             }, delay);
114           };
115
116           void (async () => {
117             let jobId: string;
118             try {
119               const resp = await processVideo(uri, cid);
120               jobId = resp.job_id;
121             } catch (err) {
122               fail(classifyError(err), "ingest.submit_error");
123               return;
124             }
125             if (run.cancelled) return;
126             log("info", "ingest.queued", { cid, uri, job_id: jobId });
127             setPhase({ status: "polling", progress: null });
128             poll(jobId, POLL_MIN_MS);
```

```
129           })();
130         },
131         [stop],
132       );
133
134       const cancel = useCallback(() => {
135         stop();
136         setPhase({ status: "idle" });
137         log("info", "ingest.cancel", {});
138       }, [stop]);
139
140       const reset = useCallback(() => {
141         stop();
142         setPhase({ status: "idle" });
143       }, [stop]);
144
145       const busy = phase.status === "submitting" || phase.status === "polling";
146       return { phase, submit, cancel, reset, busy };
147     }
148
```

22. assistant (2026-06-21T14:34:23.696Z)

Let me look at the existing SPA test files to understand the test conventions I should mirror, and check the kn/hi catalog structure to know exactly which keys must mirror.

23. user (2026-06-21T14:34:24.881Z)

No files found

24. user (2026-06-21T14:34:26.932Z)

SCRIPTS:

```
{
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview",
  "typecheck": "tsc --noEmit",
  "test": "vitest run"
}
```

DEPS:

```
{
  "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
  "@fontsource-variable/noto-sans-kannada": "^5.2.9",
  "i18next": "^26.3.1",
  "i18next-browser-languagedetector": "^8.2.1",
  "i18next-icu": "^2.4.3",
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "react-i18next": "^17.0.8"
}
```

DEVDEPS:

```
{
  "@tailwindcss/vite": "^4.0.0",
  "@testing-library/dom": "^10.4.1",
  "@testing-library/react": "^16.3.2",
  "@types/node": "^26.0.0",
  "@types/react": "^19.0.0",
  "@types/react-dom": "^19.0.0",
  "@vitejs/plugin-react": "^4.3.4",
  "jsdom": "^29.1.1",
  "tailwindcss": "^4.0.0",
  "typescript": "^5.7.0",
  "vite": "^6.0.0",
```

```
  "vitest": "^3.0.0"
}
```

25. assistant (2026-06-21T14:34:37.822Z)

```
The test glob found nothing ? let me find where the existing test files actually are (the
hygiene test is at i18n/hygiene.test.ts so tests do exist).
```

26. user (2026-06-21T14:34:39.656Z)

```
./src/api/client.test.ts
./src/components/IngestPanel.test.tsx
./src/components/QueryBar.examples.test.ts
./src/hooks/useIngestJob.test.ts
./src/i18n/hygiene.test.ts
./src/i18n/i18n.test.tsx
./src/lib/errors.test.ts
./src/lib/floor.test.ts
./src/lib/format.test.ts
./src/lib/query.test.ts
./src/lib/selection.test.ts
```

27. user (2026-06-21T14:34:41.138Z)

```
KN top keys: ['app', 'lang', 'bulk', 'mode', 'query', 'grid', 'footer', 'result', 'build',
'ingest', 'selection']
KN ingest: ['title', 'intro', 'uriPlaceholder', 'uriAriaLabel', 'hint', 'submit', 'submitting',
'cancel', 'working', 'progress', 'retry', 'failed', 'invalidUri', 'networkError', 'emptyResult',
'timeout']
KN result: ['ready', 'failed', 'dismiss', 'download']
KN build: ['noVideo']
```

28. assistant (2026-06-21T14:34:44.073Z)

```
Let me read the existing test files most relevant to my tickets: the IngestPanel test, errors
test, and the i18n render test, to mirror conventions accurately.
```

29. user (2026-06-21T14:34:44.553Z)

```
1       // @vitest-environment jsdom
2       //
3       // The ingest screen end to end: type a URI ? submit ? (mocked engine) ? onLoaded gets
the video_id;
4       // and the two failure surfaces a non-technical user actually hits ? the engine's own
400 detail,
5       // and a friendly "is the engine running?" when it's unreachable. Mounts the real i18n
catalog.
6       import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7       import { render, screen, fireEvent, cleanup, act } from "@testing-library/react";
8       import i18n from "../i18n/index";
9       import { IngestPanel } from "./IngestPanel";
10      import type { JobStatus } from "../api/types";
11
12      type FetchFn = typeof fetch;
13
14      function httpOk(status: number, json: unknown) {
15        return { ok: status >= 200 && status < 300, status, json: async () => json, text:
async () => JSON.stringify(json) };
16      }
17
18      function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
19        let i = 0;
20        return vi.fn(async (url: string | URL, init?: RequestInit) => {
21          const u = String(url);
```

```
        if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
        if (u.startsWith("/api/jobs/")) {
          const body = jobs[Math.min(i, jobs.length - 1)];
          i += 1;
          return httpOk(200, body);
        }
        throw new Error(`unexpected fetch ${u}`);
      }) as unknown as FetchFn;
    }

    beforeEach(async () => {
      window.localStorage.clear();
      await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
    });

    afterEach(() => {
      cleanup();
      vi.restoreAllMocks();
      vi.unstubAllGlobals();
      vi.useRealTimers();
    });

    const typeUri = (value: string) =>
      fireEvent.change(screen.getByLabelText(/video location/i), { target: { value } });
    const clickProcess = () => fireEvent.click(screen.getByRole("button", { name: /process
video/i }));

    describe("IngestPanel", () => {
      it("submits a URI, polls, and hands the video_id to onLoaded", async () => {
        vi.useFakeTimers();
        vi.stubGlobal(
          "fetch",
          fetchSeq(httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" }), [
            { job_id: "j", status: "running", progress: "segmenting" },
            { job_id: "j", status: "done", video_id: "vidZ", result: { status: "success",
video_id: "vidZ" } },
          ]),
        );
        const onLoaded = vi.fn();
        render(<IngestPanel onLoaded={onLoaded} />);
        typeUri("gs://bucket/match.mp4");
        clickProcess();

        await act(async () => {
          await vi.advanceTimersByTimeAsync(0); // flush submit ? start polling
        });
        await act(async () => {
          await vi.advanceTimersByTimeAsync(1000); // running tick
        });
        await act(async () => {
          await vi.advanceTimersByTimeAsync(1500); // done tick
        });
        expect(onLoaded).toHaveBeenCalledWith("vidZ");
      });

      it("shows the engine's own 400 detail (e.g. an unsupported scheme)", async () => {
        vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" }), []));
        render(<IngestPanel onLoaded={vi.fn()} />);
        typeUri("ftp://nope");
        clickProcess();
        expect(await screen.findByText(/unsupported storage URI scheme/i)).toBeTruthy();
      });

```

```
83        it("shows a friendly, localized message when the engine is unreachable", async () => {
84          vi.stubGlobal("fetch", vi.fn(async () => {
85            throw new Error("offline");
86          }) as unknown as FetchFn);
87          render(<IngestPanel onLoaded={vi.fn()} />);
88          typeUri("gs://bucket/match.mp4");
89          clickProcess();
90          expect(await screen.findByText(/can't reach the engine/i)).toBeTruthy();
91        });
92
93        it("rejects an empty URI with a hint and never calls the engine", () => {
94          const f = vi.fn();
95          vi.stubGlobal("fetch", f);
96          render(<IngestPanel onLoaded={vi.fn()} />);
97          clickProcess();
98          expect(screen.getByTestId("ingest-error")).toBeTruthy();
99          expect(f).not.toHaveBeenCalled();
100       });
101     });
102
```

30. user (2026-06-21T14:34:44.907Z)

```
1       import { describe, it, expect } from "vitest";
2       import { parseDetail, classifyError } from "./errors";
3       import { ApiError } from "../api/client";
4
5       describe("parseDetail", () => {
6         it("extracts FastAPI's {detail}", () => {
7           expect(parseDetail(JSON.stringify({ detail: "unsupported storage URI scheme"
}))).toBe(
8             "unsupported storage URI scheme",
9           );
10        });
11        it("returns the raw body when it isn't JSON or has no string detail", () => {
12          expect(parseDetail("plain text")).toBe("plain text");
13          expect(parseDetail(JSON.stringify({ other: 1 }))).toBe(JSON.stringify({ other: 1
}));
14          expect(parseDetail(JSON.stringify({ detail: { nested: true }
}))).toBe(JSON.stringify({ detail: { nested: true } }));
15        });
16      });
17
18      describe("classifyError", () => {
19        it("a network throw (ApiError with no status) ? networkError", () => {
20          expect(classifyError(new ApiError("net", { cid: "c" }))).toEqual({ code:
"networkError" });
21        });
22        it("an HTTP error with a body ? engine + the FastAPI detail", () => {
23          const err = new ApiError("x", { status: 400, body: JSON.stringify({ detail:
"unsupported storage URI scheme" }) });
24          expect(classifyError(err)).toEqual({ code: "engine", detail: "unsupported storage
URI scheme" });
25        });
26        it("a bare 5xx with no body ? networkError (friendly 'is the engine running?', not
'HTTP 500')", () => {
27          expect(classifyError(new ApiError("x", { status: 500 }))).toEqual({ code:
"networkError" });
28          expect(classifyError(new ApiError("x", { status: 502 }))).toEqual({ code:
"networkError" });
29        });
30        it("a 5xx WITH a detail still surfaces the engine's message", () => {
31          const err = new ApiError("x", { status: 500, body: JSON.stringify({ detail: "failed
to persist job record" }) });
32          expect(classifyError(err)).toEqual({ code: "engine", detail: "failed to persist job
```

```
record" });
33        });
34        it("a 4xx with no body keeps the status (a client-side mistake, not unreachable)", ()
=> {
35          expect(classifyError(new ApiError("x", { status: 404 }))).toEqual({ code: "engine",
detail: "HTTP 404" });
36        });
37        it("a non-ApiError ? engine + the stringified error", () => {
38          expect(classifyError(new Error("boom"))).toEqual({ code: "engine", detail: "Error:
boom" });
39        });
40      });
41
```

31. assistant (2026-06-21T14:34:48.121Z)

I now have complete context. Let me look at the i18n index to understand how the namespace/catalog is registered, and confirm the ApiError shape for the onBuild change.

32. user (2026-06-21T14:34:49.541Z)

```
1       // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2       //
3       // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4       //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5       //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6       //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7       //   - one negotiation order shared with the marketing site (B3);
8       //   - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9       import i18n from "i18next";
10      import { initReactI18next } from "react-i18next";
11      import LanguageDetector from "i18next-browser-languagedetector";
12      import ICU from "i18next-icu";
13      import enCommon from "./locales/en/common.json";
14      import knCommon from "./locales/kn/common.json";
15      import hiCommon from "./locales/hi/common.json";
16
17      // Supported locales (D6): `en` + `kn` + `hi`. Anything else falls back to `en`.
18      export const SUPPORTED_LOCALES = ["en", "kn", "hi"] as const;
19      export type Locale = (typeof SUPPORTED_LOCALES)[number];
20      export const DEFAULT_LOCALE: Locale = "en";
21
22      // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
23      // visitor's language choice carries across both surfaces.
24      export const LANG_STORAGE_KEY = "khelsutra.lang";
25
26      // Catalogs are co-located here (D9). `kn` + `hi` shipped (B5/B6, AI-drafted ? native
review
27      // pending). A missing key in any locale still falls back to `en` ? the by-design
invariant.
28      const resources = {
29        en: { common: enCommon },
30        kn: { common: knCommon },
31        hi: { common: hiCommon },
32      } as const;
33
34      void i18n
35        .use(ICU)
36        .use(LanguageDetector)
37        .use(initReactI18next)
38        .init({
39          resources,
```

```
40          fallbackLng: DEFAULT_LOCALE,
41          supportedLngs: [...SUPPORTED_LOCALES],
42          nonExplicitSupportedLngs: true, // en-IN -> en
43          defaultNS: "common",
44          ns: ["common"],
45          interpolation: { escapeValue: false }, // React already escapes
46          returnNull: false,
47          // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
48          detection: {
49            order: ["localStorage", "querystring", "navigator"],
50            lookupLocalStorage: LANG_STORAGE_KEY,
51            lookupQuerystring: "lang",
52            caches: ["localStorage"],
53          },
54          react: { useSuspense: false },
55        });
56
57      // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
58      // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
59      function syncDocumentLang(lng: string) {
60        if (typeof document !== "undefined") {
61          document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
62        }
63      }
64      i18n.on("languageChanged", syncDocumentLang);
65      syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
66
67      export default i18n;
68
```

33. user (2026-06-21T14:34:50.401Z)

```
9:export class ApiError extends Error {
10-  readonly cid?: string;
11-  readonly status?: number;
12-  readonly body?: string;
13-  constructor(message: string, opts: { cid?: string; status?: number; body?: string; cause?:
unknown } = {}) {
14-    super(message);
15-    this.name = "ApiError";
16-    this.cid = opts.cid;
17-    this.status = opts.status;
18-    this.body = opts.body;
19-    if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
20-  }
21-}
```

34. assistant (2026-06-21T14:35:00.980Z)

I now have everything I need to write three precise, code-grounded tickets. Let me note the key findings:

A4 findings: Interactive elements on the SPA ? QueryBar input (`py-2`), QueryBar Apply button (`py-2`), example chips (`py-1`, `text-xs` ? too small), bulk buttons (`py-1`), RallyGrid card buttons (large enough but the inner `?` is `h-5 w-5`), SelectionFooter reel-name input (`py-1.5`), Build button (`py-2`), LanguageSwitcher buttons (`py-1.5`), IngestPanel input/submit/cancel (`py-2`). Tooltips via native `title=` exist in two spots: QueryBar unavailable chips (`title={t("query.notAvailableTitle"...)}` line 71) and RallyGrid shots chip (`title={t("grid.shotsTitle"...)}` line 62) ? these are invisible on touch (no hover). The 44px min tap target is the WCAG/HIG standard.

A5 findings: `onBuild` (App.tsx:145-150) currently sets `setBuild({ status: "error", error:

detail })` where `detail = parseDetail(err.body)` for any HTTP error or `String(err)` for network ? raw FastAPI text. `classifyError` already exists and handles exactly this (5xx no body ? networkError, engine detail passthrough). Recommend a SHARED `errors.*` namespace and unify IngestPanel onto it.

A8 findings: IngestPanel `ingest.working` line + the build state (SelectionFooter `building` / CompileResult). Status renders: CompileResult title uses text only (good); needs icon+text chips. IngestPanel already has Spinner+text.

Now let me write the structured output.

35. user (2026-06-21T14:36:32.388Z)

Structured output provided successfully

36. assistant (2026-06-21T14:36:45.646Z)

I've specced the three SPA UX tickets (A4, A5, A8) and returned them via StructuredOutput. Summary of what I grounded each in:

A4 ? Mobile-first touch targets + tooltip visibility (`web/src/components/{QueryBar,RallyGrid,SelectionFooter,LanguageSwitcher,IngestPanel}.tsx` + `App.tsx`). The smallest tap targets today are the QueryBar example chips (`px-3 py-1 text-xs`, line 85-94) and the bulk buttons (`App.tsx:247-249`, `px-3 py-1`); I specced `min-h-11` (44px in Tailwind v4) per element. The two native `title=` tooltips (QueryBar unavailable chip line 71, RallyGrid shots chip line 62) are invisible on touch ? but I verified each already has a visible on-screen counterpart ("not available yet" text; "experimental" in the chip), so I recommend shipping A4 as a **pure CSS/markup PR with zero catalog edits** so it doesn't serialize with A5/A8 on the JSON.

A5 ? Friendly engine-error mapping (`web/src/App.tsx` onBuild + `IngestPanel.tsx` + catalogs). The bug: `App.tsx:145-150` shows raw `parseDetail(err.body)` / `String(err)` for compile failures (so a bare 5xx reads "HTTP 500"). The fix reuses the already-built `lib/errors.ts::classifyError`. I **recommend a shared `errors.*` namespace** (codes are flow-agnostic: `invalidUri|networkError|emptyResult|timeout|engine`) and **unifying IngestPanel onto it** ? relocating the existing kn/hi translations from `ingest.*` to `errors.*` rather than re-drafting.

A8 ? Background reassurance + verdict chips (`IngestPanel.tsx`, `CompileResult.tsx`, catalogs). Adds honest "keeps running if you leave" reassurance (no fake % ? engine reports phase words only), converts the CompileResult verdict header to icon+text chips (glyph `aria-hidden`, label stays ? never icon-only), and surfaces a building state in the main column (the building affordance is currently only the footer button).

Key cross-cutting constraint flagged on all three: **A4/A5/A8 all touch the en/kn/hi catalogs and the strict key-parity CI (`hygiene.test.ts`), so they must serialize** ? and A4 can sidestep the JSON entirely if shipped copy-free (recommended). New i18n keys, test plans (mirroring `IngestPanel.test.tsx`'s jsdom + real-i18n harness), gotchas, and effort are in the structured tickets.